

测试计划与用例文档

高校实验室资产与报修管理平台

项目名称	高校实验室资产与报修管理平台
团队编号	第 6 组
团队成员	王旻辉、巴力江·托力肯别克、廖煌、迪丽热巴·阿布都西克尔
班级 / 学号	软件工程 2023-2 / 2311231056
二级学院 / 专业	信息科学与工程学院 / 软件工程
授课教师	谭清萱
版本号	V1.0
日期	2026-09-18
编写人 / 审核人	/

本文件是源稿（Markdown）。定稿流程：AI 出初稿 → 人工改写表述（防查重）→ 补封面信息 → 导出 PDF（须嵌入字体）。

导出前请把正文中的 `_____` 与「待人工改写」标记全部清掉。

修订历史

版本	日期	修订内容	修订人
V0.1	_____	初稿完成（AI 辅助生成，全部数字取自测试构建日志）	_____
V1.0	_____	人工改写表述、补齐封面信息、定稿	_____

编写说明与 AI 使用声明

本文档允许使用 AI 工具，但内容的准确性由本组负责。各章的生成方式如实声明如下：

章节	生成方式	说明
1 引言~2 测试策略	AI 生成初稿 → 待人工改写	策略与比例为本组实际执行的口径
3 环境与运行方式	AI 生成初稿 → 待人工改写	命令为真实执行的命令，三前提为实测结论
4 单元测试~6 等价类与边界值	AI 生成初稿 → 待人工改写	用例清单逐条对应 <code>src/test</code> 真实类与方法
7 追溯矩阵~8 测试结果	AI 生成初稿 → 待人工改写	数字全部来自 Maven 构建日志，可逐条复核
9 限制与如实说明	AI 生成初稿 → 待人工改写	覆盖率等不利事实照实列出，未作粉饰

防查重提示（提交前请删除本条）：本稿依据 `AI留痕/测试素材交付.md`、T1/T2/T2b 留痕与构建日志组织，与内部文件存在同源表述风险。定稿前必须人工通读重写并查重。

1 引言

1.1 编写目的

本文档给出「高校实验室资产与报修管理平台」的测试策略、测试环境、用例设计方法、用例与业务规则的对应关系，以及最终测试结果。所有数字均来自真实执行的 Maven 构建日志（`Tests run` 汇总与逐类耗时），不存在估算值。

1.2 测试范围

覆盖自定义业务包 `com.ruoyi.project.laboratory` 的四个模块：实验室房间、资产台账、故障报修（含状态机与履历）、数据看板。若依框架原生模块（系统管理、监控等）不在本项目测试范围内。

1.3 通过标准

全量用例 100% 通过（Failures = 0、Errors = 0、Skipped = 0）；每条业务规则 BR-01~BR-06 至少被一个用例的断言命中，且断言对象可指回规则编号。

2 测试策略与测试金字塔

2.1 比例与理由

层	用例数	占比	为什么这么分
单元测试	33	67%	报修状态机、权限判定、字段校验三块规则密集、分支多，需要毫秒级快速反馈；用 Mockito 隔离依赖，专攻「调用契约」
集成测试	16	33%	本系统一次写操作常常同时落 2~3 张表（报修单 + 资产状态 + 履历），事务一致性是单测用 Mock 验证不了的；集成测试用真实 SQL 验证「落库结果」

层	用例数	占比	为什么这么分
端到端	2 条主流程	手工执行	只保留「提交报修→审核→完成」「资产台账增删改查」两条演示主流程，由人工在真实环境按步骤执行

关于端到端占比低：本项目没有引入 Selenium / Playwright，原因有二：一是离线开发环境无法安装浏览器驱动，二是系统体量（4 个业务模块）不足以摊薄 E2E 框架的维护成本。端到端场景由人工按演示路径实测替代（登录、看板、台账、报修闭环四条路径共 73 条接口请求，0 个 4xx/5xx）。这是有意的设计取舍，不是遗漏。

2.2 单元与集成的分工边界

维度	单元测试	集成测试
验证方式	Mockito 隔离依赖，验证方法被调几次、参数对不对	Spring 上下文 + 真实数据库（H2 内存库），JdbcTemplate 直查表验证落库结果
能发现	分支遗漏、边界值错误、异常消息错误、调用顺序错误	多表事务不一致、Mapper XML 与表结构不匹配、SQL 方言问题、行级权限过滤失效
发现不了	SQL 是否真的执行成功	单个方法的边界组合（集成测试慢，不适合穷举）
中间件依赖	无	无（H2 内存库，本地跑测试不需要 MySQL / Redis / Docker）

3 测试环境与运行方式

3.1 运行命令

```
# 本机首选（Maven 启动脚本损坏时走包装脚本）
powershell -NoProfile -ExecutionPolicy Bypass -File .workbuddy/tools/mvnx.ps1 -o -B clean test

# Git Bash 环境等价命令
bash .workbuddy/tools/mvnx.sh -o -B clean test

# 只跑集成测试（调试用）
powershell ... mvnx.ps1 -o -B -Dtest='*IntegrationTest' test
```

3.2 环境三前提（缺一条无法复现）

#	前提	原因
1	必须离线 -o	Maven 本地仓库已预热完毕，离线模式保证构建可重复
2	H2 版本必须锁 1.4.199	Spring Boot 2.5.15 依赖管理指向 1.4.200，本机离线仓库没有该版本；不锁版本会报 Could not resolve dependencies，极易误判为网络问题。已在 pom.xml 以 test scope 锁定
3	不需要 Docker / Testcontainers / 嵌入式 Redis	本机离线仓库没有这些组件；集成测试全部跑 H2 内存库

3.3 两个踩坑记录

- 验收一律用 clean test：不带 clean 时 target/test-classes 会残留上一次编译的类，导致 mvn test 与 clean test 结果不一致——项目在对照实验中因此失效过一轮。
- 集成测试不加 @Transactional：测试类用 @BeforeEach 清表 + 铺基线数据，保证用例间互不污染，同时避免事务回滚掩盖「落库是否真实发生」这一被测对象本身。

4 单元测试设计（33 个）

4.1 用例分布

测试类	用例数	被测对象	考察点
LabRepairServiceImplTest	12	报修服务	状态机合法/非法流转全枚举、字段校验、资产联动、评分边界
LabAssetServiceImplTest	8	资产服务	唯一性校验、删除保护、批量导入的成败计数
LabRepairStatusMachinePublicEntryTest	3	报修公开入口	状态机经 <code>updateLabRepair</code> 真实入口的分支与持久化编排
LabRoomServiceImplTest	5	房间服务	房间删除保护、编号与名称必填、编号占用
LabRoleUtilsTest	5	权限判定	六类角色的处理权/全局可见权矩阵、无登录上下文兜底

4.2 状态机的两层覆盖

状态机是全系统风险最高的规则（写错会静默放行非法操作），因此覆盖分两层：

- 契约层（保留的 5 个反射用例）：直接验证 4 条合法流转 + 16 组非法流转 + 幂等（同状态放行）矩阵。缺点是跨过了公开入口。
- 入口层（新增 3 个用例）：经 `updateLabRepair` 公开入口触发，补上契约层测不到的三件事——合法流转时持久层真的收到正确参数、非法流转时整轮零写库（`verify(..., never())`）、状态不变时履历记的是「维修信息更新」而非状态流转。

两层并存是有意决策：契约层穷举矩阵、入口层验证编排，互不弱化。

5 集成测试设计（16 个）

5.1 用例分布

测试类	用例数	覆盖
LabRepairIntegrationTest	7	报修全流程落库、非法流转三处不变、行级权限、逻辑删除、终态不可回退/复活
LabDashboardIntegrationTest	7	看板 8 条 mapper 查询全部覆盖（4 项指标 + 4 张图表），含 7 天窗口边界
LabAssetIntegrationTest	2	跨模块协作（资产↔房间）、批量导入事务与履历

5.2 看板方言垫片（不改生产代码补齐 H2 方言）

看板 SQL 使用 MySQL 专有的 `date_format()` / `date_sub()`，H2 不支持。本项目的解法：在测试侧用 H2 的 `create alias ... for "类.静态方法"` 注册两个同名等效函数，生产 mapper XML 一个字都没改就在 H2 上跑通了全部 8 条查询。垫片放在 `src/test`，不参与生产打包。

一个实现细节值得记录：`date_sub` 的第二个参数必须声明成 `String`——H2 会把 `interval 6 day` 变成 `INTERVAL '6' DAY` 传进来；声明成 `int` 报数据转换错误，声明成 `Object` 报十六进制解析错误。垫片自检用例（DB-01）验证的是函数真的算得出正确值，而不是只验证注册记录存在。

5.3 基础设施自检与数据铺底

每个集成测试类自带 `@BeforeEach` 基线（房间/资产/报修数据），不依赖执行顺序；DB-01 作为基础设施自检用例单独存在，方言垫片失效时第一个红的就是它，便于定位。

6 黑盒用例设计方法：等价类与边界值

6.1 等价类划分（有效/无效各一组）

类型	实例	落在哪个用例
有效等价类	资产状态=正常（可报修）；报修状态=待审核（可删）；房间下资产数=0（可删房间）；已完成工单（可评价）	IT-01、IT-04、IT-05、 <code>LabRepairServiceImplTest</code>
无效等价类	待审核直接跳已完成；房间下有资产时删房间；资产编号已被占用；待审核单直接评价	IT-03、IT-05、IT-07、 <code>LabRepairServiceImplTest</code>

6.2 边界值选取（七组）

边界对象	取值	用例
评分区间 [1,5]	0 / 1 / 5 / 6（1、5 为边界内，0、6 为边界外）	<code>LabRepairServiceImplTest</code>
必填字符串	空串与 <code>null</code> 分开各测	<code>LabRepairServiceImplTest</code>
批量导入条数	0 条（空清单）与 3 条含 1 条失败，配成一对	<code>LabAssetServiceImplTest</code> 、IT-07
近 7 天窗口	第 6 天计入、第 7 天剔除	DB-04、DB-07
排行榜上限	<code>limit 8</code> 满载与未分配实验室兜底	DB-06
权限身份	本人 / 另一普通用户 / 处理角色 / 无登录上下文	IT-06、 <code>LabRoleUtilsTest</code>
状态机终态	3→2（不可回退）、4→0（不可复活）、0→3（跨级跳转）	IT-03、IT-09、IT-10

7 用例 ↔ 业务规则追溯矩阵

编号口径取自需求基线的 BR-01~BR-06。一个用例可命中多条规则（如全流程用例 IT-02 同时命中 BR-01/02/04），故各行合计 58 大于用例总数 49。

规则	用例数	代表性断言点
BR-01 报修状态机	12	4 合法+16 非法全枚举；公开入口版本；全流程每步落库；终态不可回退/复活
BR-02 资产状态联动	5	提交→资产变维修中；完成→回归正常；删除待审核单→回滚；拒绝单不占资产
BR-03 权限规则	8	六角色矩阵；行级过滤真实生效（同一数据两种身份见不同范围）；看板口径按角色收紧
BR-04 事务边界	6	批量导入逐条独立提交；非法流转零写库；报修单+资产+履历三表同写；失败整体不变
BR-05 字段校验	20	报修 6 项 + 资产 8 项 + 房间 5 项 + 集成侧撞号汇总（矩阵中最大的一块）
BR-06 逻辑删除	2	删除后物理行仍在、 <code>del_flag='2'</code> ；清空资产后房间可删
US-13 运维看板（用户故事）	4	7 天窗口边界、等级分布、排行 <code>limit</code> 、成本趋势窗口
基础设施自检	1	方言垫片真的算得出正确值

断言总数 177 条（`assertEquals` / `assertThrows` / `assertTrue` / `assertNotNull` / `assertDoesNotThrow`），Mockito 交互校验 27 次 `verify`——后者专门用来断言「拒绝时确实没有写库」这类否定式契约，是 `assertThrows` 覆盖不到的。

8 测试结果与证据

8.1 汇总数字（全部来自真实构建日志）

指标	任务书要求	实际值
单元测试数	≥10	33
集成测试数	≥5	16
合计通过	—	49 / 49 = 100% (Failures 0 / Errors 0 / Skipped 0)
单次全量耗时	—	1 分 38 秒
断言总数	—	177

指标	任务书要求	实际值
Mockito 交互校验	—	27
集成测试中间件	—	0 个（H2 内存库）
业务规则覆盖	—	BR-01~06 全部命中（58 条映射）

8.2 逐类结果

测试类	类型	用例	耗时
LabAssetIntegrationTest	集成	2	8.101 s
LabDashboardIntegrationTest	集成	7	0.194 s
LabRepairIntegrationTest	集成	7	0.313 s
LabAssetServiceImplTest	单元	8	0.228 s
LabRepairServiceImplTest	单元	12	0.043 s
LabRepairStatusMachinePublicEntryTest	单元	3	0.002 s
LabRoomServiceImplTest	单元	5	0 s
LabRoleUtilsTest	单元	5	0 s

8.3 测试运行证据（逐字摘录自构建日志）

证据一 · 汇总行：

```
Tests run: 49, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

证据二 · 看板 SQL 在 H2 上真实执行（生产 mapper XML 原文，一字未改）：

```
==> Preparing: select date_format(create_time, '%m-%d') as name, count(1) as value
      from lab_repair where del_flag = '0'
      and create_time >= date_sub(curdate(), interval 6 day)
      group by date_format(create_time, '%m-%d') order by min(create_time)
<==      Total: 3
```

Total: 3 正是期望的「今天 / 3 天前 / 6 天前」三个分组——窗口外的第 7 天那条被正确剔除，窗口边界行为与断言一致。

[待填：此处插入测试运行截图 1 — 用浏览器打开 AI留痕/截图/T2b-测试运行结果-逐字摘录.html 截取，存为 PNG 后替换本占位]

[待填：此处插入测试运行截图 2 — 需求同上，建议截取含 Tests run 汇总行与 BUILD SUCCESS 的段落]

逐字摘录的原始凭据：AI留痕/截图/T2b-Tests-run-汇总行.txt 与完整构建日志。截图须人工从上述文件截取生成，不得以文本文件代替截图提交。

9 限制与如实说明

以下四条是本测试工作的已知边界，均照实说明、未作美化：

1. 行覆盖率 / 分支覆盖率百分比未能产出。 本项目全程离线构建，本地 Maven 仓库缺 jacoco-maven-plugin 与 org.jacoco.core，仅有运行时 agent jar，无法生成覆盖率报告；引入 JaCoCo 还需修改 pom.xml 构建段，超出测试工作的改动边界。因此本文以用例↔业务规则追溯矩阵（第 7 章）作为需求维度的覆盖证据，辅以断言数 177 与交互校验 27 两个可机械复核的统计量，不给出未经实测的覆盖率百分比。 2. 构建日志含一段无害噪声。 Surefire 2.22.2 的 PpidChecker 会调用 Windows WMIC 查询父进程存活状态，本机安全策略拦截了 WMIC，Surefire 随即自行降级为 NOOP events 并留下一段 IllegalStateException: Cannot use PPID ... 堆栈与一个

`jvmRun1.dump` 文件。该噪声不影响测试：本轮 49/49 通过、BUILD SUCCESS。在此说明是为了避免读者把这段堆栈误读为构建故障。

3. 事务回滚为间接覆盖。「注入异常后断言双表回滚」的直接验证未做：本机离线仓库中 MyBatis Mapper 是 JDK 动态代理，`@SpyBean` 对其生效性存疑，硬造一个假回滚断言不如如实说明。事务完整性目前由 IT-03「非法流转后单据、履历、资产三处均不变」间接覆盖。

4. 测试聚焦 Service 与 Mapper 两层。29 个生产类中被测试直接触达的核心类 11 个；Controller 层的权限注解通过真实环境端到端实测验证（四条演示路径 73 条接口 0 个 4xx/5xx），Entity 与纯常量/图形工具类不单独编写用例。